MetaboTrack API: Architectural Design and Analytical Implementation

1. Project Links and Overview

1.1 Project Links

In accordance with the deliverable requirements , the following resources are publicly accessible:

- **GitHub Repository:** MetaboTrack-API GitHub Repository
- **API Documentation (PDF):** MetaboTrack-API - API_Documentation
- **Presentation Slides:** MetaboTrack API - Presentation Slides

1.2 System Overview

The MetaboTrack API is a data-driven web service. The system is designed to manage and interpret a complex physiological dataset containing 15,000+ daily metabolic records (Parashar, 2026).

Unlike conventional "Data Sink" applications that merely provide basic CRUD functionality, MetaboTrack is engineered as an Analytical Insight Engine. It addresses the challenge of large-scale health data management by providing sophisticated endpoints for:

1. **Macro-level Population Insights:** Utilizing advanced SQL aggregations to identify trends in metabolic efficiency and cardiovascular health across the entire dataset.
2. **Predictive Simulation:** Implementing a deterministic, rule-based engine that allows users to project future health outcomes based on hypothetical lifestyle adjustments.
3. **Performance Ranking:** Computing real-time, individual percentile rankings to enhance user engagement through data-driven feedback.

The system adopts a robust, layered architecture utilizing Spring Boot 3.x and MyBatis-Plus, ensuring high scalability and maintainability while maintaining a focus

on performance optimization for large-sample data analysis.

2. Technology Stack & Architectural Justification

2.1 Technology Stack Justification

To ensure the MetaboTrack API is robust, scalable, and capable of processing over 15,000 physiological records, the following technology stack was deliberately selected:

- **Java 17 & Spring Boot 3:** Chosen for its robust object-oriented paradigm, strong type safety, and the powerful Inversion of Control (IoC) container. Spring Boot significantly accelerates backend development by providing embedded web servers and auto-configuration, allowing the focus to remain on complex business logic rather than boilerplate setup.
- **MySQL 8:** A relational database is the optimal choice for highly structured, tabular

physiological data (e.g., steps, heart rate, calories). Furthermore, strategic B-Tree indexing on columns like `user_id` and `calorie_efficiency` was implemented to reduce query time complexity from $O(N)$ (full table scans) to $O(\log N)$, ensuring rapid pagination and aggregation.

- **MyBatis-Plus & Hybrid SQL Strategy:** A conscious hybrid approach was adopted for data access. For standard CRUD operations, MyBatis-Plus's `LambdaQueryWrapper` was utilized to guarantee compile-time type safety and prevent SQL injection. However, for advanced analytical endpoints, ORM abstractions were intentionally bypassed in favor of native `@Select` SQL queries to maximize the database engine's aggregation performance.

```
Page<DailyMetabolicRecord> resultPage = lambdaQuery()
        .eq( condition: recordPageQueryDTO.getUserId() != null, DailyMetabolicRecord::getUserId, recordPageQueryDTO.getUserId())
        .eq( condition: recordPageQueryDTO.getCalorieEfficiency() != null, DailyMetabolicRecord::getCalorieEfficiency, recordPageQueryDTO.getCalorieEfficiency())
        .orderByDesc(DailyMetabolicRecord::getEfficiencyScore)
        .page(page);
```

```
@Select("SELECT COUNT(*) FROM (" +
        "  SELECT user_id, AVG(efficiency_score) as avg_score " +
        "  FROM daily_metabolic_record " +
        "  GROUP BY user_id " +
        "  HAVING avg_score < #{targetScore}" +
        ") AS subquery")
Long countUsersBelowScore(@Param("targetScore") Double targetScore);
```

Figure 1. *Screenshots of MP's LambdaQuery and @Select SQL query*

2.2 Architectural Design Decisions

The application strictly adheres to the Controller-Service-Mapper layered architecture to guarantee the Separation of Concerns (SoC). Beyond this standard structure, two critical architectural patterns were implemented to elevate the system to an enterprise-grade standard:

1. JWT-Based Authentication & Data Isolation (IDOR Defense)

```
@Override
public void addInterceptors(InterceptorRegistry registry
    registry.addInterceptor(loginInterceptor)
            .addPathPatterns("/api/**")
            .excludePathPatterns(
                    "/api/user/login",
                    "/api/user/register",
                    "/swagger-ui/**",
                    "/v3/api-docs/**"
            );
}
```

```
if (!record.getUserId().equals(UserContext.getCurrentId())) {
    return Result.error( code: 403,
            msg: "Access Denied: You do not have permission to access this record.");
}
```

```
// Store the user ID in the ThreadLocal context
UserContext.setCurrentId(userId);
```

Figure 2. *Screenshots of Interceptor and ThreadLocal context*

The system has moved away from insecure `userId` parameter passing to a robust JWT-based stateless authentication. Utilizing a custom Interceptor and `ThreadLocal` context, user identity is securely extracted from the Authorization header. In the `Record` and `Analytics` modules, all write operations and sensitive queries are strictly bound by `WHERE user_id = current_user_id` logic, effectively neutralizing Horizontal Privilege Escalation (IDOR) at the architectural level.

2. Database Engine Offloading for Macro-Insights

When dealing with a dataset of 15,000+ rows, performing data aggregations (e.g., calculating population percentile distributions or sleep quality leverage) within the Java JVM using loops leads to severe memory overhead and risks `OutOfMemoryError`. To mitigate this, complex analytical logic was pushed down to the database layer. By utilizing advanced SQL features like `GROUP BY`, nested subqueries, and `CASE WHEN` for Data Binning (e.g., categorizing continuous sleep hours into

discrete buckets), the application achieves an $O(1)$ memory space complexity, transferring only the final aggregated metrics across the network.

3. Dynamic Simulation and Predictive Engine

The core innovation of the MetaboTrack API is the Dynamic Predictor and Simulator (`/api/analytics/simulator`). While conventional endpoints describe historical states, this engine functions as a proactive tool for behavioral health modeling, allowing users to project future health outcomes based on hypothetical lifestyle adjustments.

3.1 Deterministic Rule-Based Design

The simulator is architected as an in-memory Deterministic Rule-Based Engine. By strictly implementing the dataset author's physiological heuristics(Parashar, 2026), it accurately models the interplay between lifestyle variables and metabolic outcomes. Executing this logic entirely within Java RAM offers two critical advantages:

1. **Latency Optimization:** Simulations are computed in $O(1)$ time, enabling instantaneous "what-if" feedback for interactive UI components without the latency of database I/O.
2. **Prescriptive Value:** The logic is isolated from the persistence layer, allowing the system to project hypothetical scenarios that do not yet exist in the database, thereby guiding users toward healthier habits.

2.1 Explicit Calculation Heuristics & Formulas

Phase	Metric	Formula / Adjustment Rule
1. Base Signal	Core Efficiency	$Base = \frac{Calories}{Steps + (20 \times ActiveMinutes)}$
2. Linear Weights	Muscle Mass	$+0.3$
	Body Fat	-0.2
	Hydration	$+0.05$ per liter
	Sleep Hours	$+0.05$ per hour
1. Non-Linear Multipliers	Resting HR Scaler	$Score = Score \times \left(\frac{80}{RestingHR}\right)$
	Average HR Scaler	$Score = Score \times \left(\frac{120}{AverageHR}\right)$
2. Consistency & Penalties	Habit Bonus	$+3\%$ per continuous day ($Score \times 1.03$)
	Adaptation Jump	$+10\%$ flat jump upon crossing the 5-day streak
	Over-training Drop	-15% penalty for workouts > 6 days/week
	Exhaustion Penalty	-10% penalty if streak ≥ 6AND sleep < 6 hours
	Severe Deprivation	-25% flat drop if sleep < 5hours
3. Boundary Enforcement	Numerical Limits	Math.min(10.0, Math.max(0.0, Score))

Table 1. Efficiency Calculation Rules

To achieve high physiological fidelity, the engine translates the dataset's underlying logic into a

strict, multi-phase mathematical model. The complete calculation pipeline is defined in the heuristic table above

The continuous score is finally mapped to categorical tiers: High Efficiency (≥ 7.0), Moderate ($4.0 - 6.9$), and Low Efficiency (< 4.0). This deterministic approach demonstrates a deep understanding of domain-specific business logic beyond simple data storage.

```
// 1. The Base Signal
// efficiency = calories_burned / (steps_per_day + 20 * active_minutes)
double denominator = req.getTargetSteps() + (20.0 * req.getTargetActiveMinutes());
double baseSignal = denominator > 0 ? (req.getTargetCaloriesBurned() / denominator) * 100.0 : 0.0;
double score = baseSignal;
feedback.add("Base signal calculated from movement-to-calorie ratio.");

// 2. Linear Weights
// Hydration: +0.05 weight | Sleep: +0.05 weight
score += (req.getTargetHydration() * 0.05);
score += (req.getTargetSleepHours() * 0.05);
```

Figure 3. *Efficiency calculation rule code (partial)*

4. Testing Approach & Error Handling

The reliability and usability of the MetaboTrack API are ensured through a centralized error-handling architecture and an interactive documentation-driven testing strategy.

1. **Global Exception Handling and Standardized Responses:** To maintain a robust and secure API contract, scattered `try-catch` blocks were avoided. Instead, a global interceptor utilizing `@RestControllerAdvice` was implemented. This mechanism catches both custom `BusinessException` instances (e.g., "Record Not Found") and unexpected runtime errors globally. It guarantees that the client always receives a predictable, standardized JSON wrapper (`Result<T>`) containing appropriate HTTP status codes and actionable error messages, strictly preventing internal Java stack traces from leaking to the frontend.
2. **Interactive Documentation-Driven Testing:** Springdoc OpenAPI (Swagger 3) was integrated not merely as static documentation, but as the primary interactive testing suite. By leveraging the `@Schema(example="...")` annotation across all DTOs and Controllers, every endpoint is pre-populated with valid JSON payloads and path variables. This design choice drastically lowers the barrier to entry, allowing developers to conduct immediate, successful integration tests directly from the browser without requiring external tools like Postman.

5. Limitations & Future Development

While the API efficiently fulfills the coursework requirements and successfully manages the 15,000+ record dataset, certain architectural limitations exist that would require mitigation before a production-grade release.

1. **Stateless Logout Limitation:** Although IDOR vulnerabilities have been remediated, the current logout process is entirely stateless (relying on client-side disposal), meaning the backend cannot actively invalidate issued tokens. Furthermore, the manual interceptor-based security logic is less robust than industry-standard frameworks.
   - **Future Development** plans to integrate Spring Security to provide a more comprehensive security infrastructure. Additionally, a Redis-based Blacklist will be explored to store invalidated tokens with a matching TTL upon logout, enabling true server-side token

revocation.

2. **High-Concurrency Database Bottlenecks:** Although offloading macro-analytics to the database resolves JVM memory constraints, repeatedly executing complex nested subqueries (e.g., the percentile ranking engine) under high concurrent traffic would inevitably strain the MySQL CPU.
   - **Future development** would introduce an in-memory caching layer using Redis. Expensive macro-analytics and leaderboards would be cached and scheduled to refresh periodically (e.g., hourly) using Spring Cache, drastically reducing direct database I/O and ensuring sub-millisecond response times.

6. Generative AI Declaration & Usage Analysis

In accordance with the "Green Light Assessment" policy, I formally declare the use of Generative AI tools (Google Gemini 3.1 Pro) throughout the lifecycle of this coursework. Full conversation logs are attached as supplementary material.

My engagement with Generative AI involved architectural-level discussion and technical implementation assistance, demonstrating creative application across four key domains:

1. **Construction of the Data Ingestion Pipeline**

   I utilized GenAI to assist in developing an efficient Python script to populate the database with the 15,000-record dataset. We discussed optimizing the I/O process, which led to the implementation of an in-memory Enum Mapping strategy. This approach converted categorical string labels into integers prior to database insertion, significantly reducing the B-Tree index footprint and accelerating future query speeds.

2. **Optimization of Backend SQL Aggregation & Database Offloading**

   A significant portion of my dialogue involved debating the space complexity trade-offs of macro-analytics. We explored the performance implications of fetching records into the Java JVM versus pushing computation down to the MySQL engine. This led to the architectural decision to implement In-Database Aggregations via nested subqueries, ensuring an $O(1)$ memory footprint for the Spring Boot application and preventing `OutOfMemoryError` under high concurrency.

3. **Mathematical Dissection of the Simulator Algorithm**

   The development of the `/simulator` endpoint required a rigorous mathematical translation of the physiological heuristics explicitly provided by the dataset author. I utilized GenAI to help dissect these abstract behavioral descriptions into a deterministic mathematical pipeline. This collaborative analysis ensured that the Java-based Rule Engine maintained perfect algorithmic fidelity to the author's specified generation logic while remaining computationally efficient.

4. **Professional API Documentation Engineering**

   To ensure the API deliverables met publication-quality standards, I collaborated with GenAI to craft precise, industry-standard descriptions and structural schemas for the Springdoc OpenAPI (Swagger 3) integration. The AI assisted in formulating complex `@Schema(example="...")` JSON payloads for various DTOs, lowering the barrier to entry and enabling interactive, documentation-driven testing directly from the browser.

References

[1]. Parashar, M., 2026. *Close to realistic Calorie efficiency dataset*. [dataset] Kaggle. Available at: https://www.kaggle.com/datasets/parasharmanu/close-to-realistic-calorie-efficiency-dataset [Accessed 10 April 2026].

Appendix A: Generative AI Conversation Logs

Session 1: Data Ingestion Pipeline & Storage Optimization

User:

I need to write a Python script to import a dataset of 15,000 records from a file named calorie_efficiency_dataset.csv into my MySQL database. The database has two tables: user_profile and daily_metabolic_record. One of the columns in the CSV is calorie_efficiency, which contains string labels like 'low efficiency', 'moderate', and 'high efficiency'. Can you help me set up a basic script using pandas and mysql.connector?

AI:

I can certainly help you build this data ingestion script. Using pandas to parse the CSV and mysql.connector to interface with the database is a standard and reliable approach.

However, before I draft the code, I want to address a potential architectural bottleneck. You mentioned the calorie_efficiency column contains string labels. Because you are building an analytical API, do you plan to frequently query, filter, or group by this specific column?

User:

Yes, I will be writing several REST API endpoints that aggregate macro-data and group users based on these efficiency tiers. If I just store all 15,000 strings as VARCHAR in MySQL, how will it affect my API's query speed and indexing?

AI:

Storing 15,000 categorical strings as VARCHAR will negatively impact your API's performance in two ways:

1. **Query Overhead:** Database string comparisons (character by character) are computationally more expensive than integer comparisons.
2. **B-Tree Index Bloat:** If you index this column to speed up your GROUP BY aggregations, a VARCHAR index will consume significantly more RAM and disk space. A larger B-Tree means more page fetches, slowing down traversal times.

To optimize the I/O process and future query speeds, I strongly recommend implementing an **in-memory Enum Mapping strategy**. We can create a mapping dictionary in Python to convert these categorical string labels into TINYINT integers (e.g., 1, 2, 3) *before* they hit the database.

Furthermore, instead of inserting 15,000 rows one by one, we should leverage cursor.executemany() to batch insert the records, bypassing network I/O bottlenecks.

User:

That makes perfect sense. Let's act like a Database Architect here. I want to go with the Enum Mapping strategy and batch processing.

Please generate the complete Python script. It needs to:

1. Include an EFFICIENCY_MAP dictionary for the strings.
2. Insert a user_profile first, grab the generated user_id, and then prepare the daily_metabolic_record.

3. Batch insert the metabolic records every 500 rows using executemany.
4. Fix a potential typo in the CSV where 'body_fat_percent' might need to be renamed to 'body_fat_percentage'.

AI:

Excellent architectural decision. Pushing the data transformation into the Python memory layer will keep your MySQL database lean and fast.

Here is the complete, optimized data ingestion pipeline incorporating your mapping and batching requirements:

Python

```python
import pandas as pd
import mysql.connector
import os
from datetime import datetime

# ==========================================
# 1. Configuration & Database Setup
# ==========================================
DB_CONFIG = {
    'host': '127.0.0.1',
    'user': 'root',
    'password': '@#lurn12138',  # Ensure this matches your MySQL password
    'database': 'metabotrack_db'
}

CSV_FILE_NAME = 'calorie_efficiency_dataset.csv'
MAX_RECORDS = 15000
SNAPSHOT_DATE = datetime(2026, 4, 7).date()

# ==========================================
# Data Mapping Dictionary
# Maps string categories to integer values for optimized DB storage
# ==========================================
EFFICIENCY_MAP = {
    'low efficiency': 1,
    'moderate': 2,
    # Adding 'moderate efficiency' just in case data has variations
    'moderate efficiency': 2,
    'high efficiency': 3
}


def main():
```

```python
print("Starting cross-sectional data pipeline with Enum Mapping...")

current_dir = os.path.dirname(os.path.abspath(__file__)) if '__file__' in locals() else os.getcwd()
csv_path = os.path.join(current_dir, CSV_FILE_NAME)

if not os.path.exists(csv_path):
    raise FileNotFoundError(f"Cannot find '{CSV_FILE_NAME}' in: {current_dir}")

print(f"Loading {MAX_RECORDS} rows from CSV...")
df = pd.read_csv(csv_path, nrows=MAX_RECORDS)
df.columns = df.columns.str.strip().str.lower()

if 'body_fat_percent' in df.columns and 'body_fat_percentage' not in df.columns:
    df.rename(columns={'body_fat_percent': 'body_fat_percentage'}, inplace=True)

print("Connecting to MySQL Database...")
try:
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()

    insert_user_sql = """
      INSERT INTO user_profile
      (age, bmi, body_fat_percentage, muscle_mass_ratio, workouts_per_week, create_time, update_time)
      VALUES (%s, %s, %s, %s, %s, %s, %s)
    """

    insert_record_sql = """
      INSERT INTO daily_metabolic_record
      (user_id, record_date, steps_per_day, active_minutes, calories_burned,
       sleep_hours, hydration_liters, heart_rate_resting, heart_rate_avg,
       continuous_exercise_days, efficiency_score, calorie_efficiency,
       create_time, update_time)
      VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """

    users_created = 0
    records_values = []
    now = datetime.now()

    print("Processing rows and mapping string values to integers...")

    for index, row in df.iterrows():
        user_values = (
            int(row['age']),
            float(row['bmi']),
            float(row['body_fat_percentage']),
            float(row['muscle_mass_ratio']),
            int(row['workouts_per_week']),
            now,
```

```
            now
        )
        cursor.execute(insert_user_sql, user_values)
        user_id = cursor.lastrowid
        users_created += 1

        # Convert string to lowercase, strip whitespace, and map to integer (default to 1 if unknown)
        raw_eff_str = str(row['calorie_efficiency']).strip().lower()
        mapped_eff_value = EFFICIENCY_MAP.get(raw_eff_str, 1)

        records_values.append((
            user_id,
            SNAPSHOT_DATE,
            int(row['steps_per_day']),
            int(row['active_minutes']),
            float(row['calories_burned']),
            float(row['sleep_hours']),
            float(row['hydration_liters']),
            float(row['heart_rate_resting']),
            float(row['heart_rate_avg']),
            int(row['continuous_exercise_days']),
            float(row['efficiency_score']),
            mapped_eff_value,  # Using the mapped integer here!
            now,
            now
        ))

        if users_created % 500 == 0:
            cursor.executemany(insert_record_sql, records_values)
            conn.commit()
            print(f"Progress: Processed and committed {users_created} users and records...")
            records_values = []

    if records_values:
        cursor.executemany(insert_record_sql, records_values)
        conn.commit()

    print(f"Data seeding completed! Imported {users_created} records with Enum Mapping optimization.")

except mysql.connector.Error as err:
    print(f"Database Error: {err}")
    if 'conn' in locals() and conn.is_connected():
        conn.rollback()
finally:
    if 'cursor' in locals():
        cursor.close()
    if 'conn' in locals() and conn.is_connected():
        conn.close()
```

```
        print("MySQL connection closed.")


if __name__ == "__main__":
    main()
```

Session 2: Space Complexity Trade-offs & Database Offloading

User:

I am building a macro-analytics endpoint: /analytics/users/{userId}/percentile. It needs to calculate a specific user's percentile ranking based on their efficiency_score compared to the entire population of 15,000 records. My initial thought is to use MyBatis-Plus to fetch all 15,000 records into a List<DailyMetabolicRecord> in Spring Boot, sort the list by score in Java, find the user's index, and calculate the percentile. Is this a sound architectural approach?

AI:

While fetching the records and sorting them in Java will work functionally, it is a well-known anti-pattern for macro-analytics and introduces severe scalability risks. Let's analyze the Big-O space complexity trade-offs here.

If you fetch 15,000 rows into the Java JVM, you are instantiating 15,000 entity objects in memory for a single request. This is an $O(N)$ space complexity. If 100 concurrent users request their leaderboard percentile at the same time, your Spring Boot application will suddenly load 1.5 million objects into the heap space. This massive memory footprint will cause frequent Garbage Collection (GC) pauses and will inevitably lead to an OutOfMemoryError under high concurrency.

User:

I see the bottleneck now. I definitely need to prevent an OutOfMemoryError and ensure the API remains highly available. If I shouldn't bring the data to the code, what is the best alternative to reduce the space complexity?

AI:

The optimal architectural decision is **Database Offloading**. You should push the computation down to the MySQL engine. Relational databases are fundamentally designed and highly optimized for sorting and aggregating massive datasets, especially if you have a B-Tree index on the efficiency_score column.

Instead of bringing 15,000 rows into Java, you can use **In-Database Aggregations** via nested subqueries. You tell MySQL to calculate the rank, and MySQL only returns a single scalar value (the final percentage) back to your Spring Boot application. This strategy guarantees a strict **$O(1)$ memory footprint** for your Java service, completely eliminating the OOM risk regardless of how large the dataset grows.

User:

That is exactly the architectural design I want to implement. I will bypass the ORM abstractions here and write a native @Select query. Can you help me formulate the nested SQL subqueries to calculate this percentile dynamically within the database?

AI:

Certainly. To achieve this entirely within MySQL, we can use a nested subquery structure in your Mapper interface. Here is the SQL logic that pushes the computation down to the database:

Java

```java
@Select("""
  SELECT
    (SELECT COUNT(*) FROM daily_metabolic_record WHERE efficiency_score <
      (SELECT efficiency_score FROM daily_metabolic_record WHERE user_id = #{userId})
    ) * 100.0 / (SELECT COUNT(*) FROM daily_metabolic_record) AS percentile
""")
Double calculateUserPercentile(@Param("userId") Long userId);
```

By executing this, the JVM memory only ever holds a single Double value ($O(1)$), while MySQL handles the heavy lifting of traversing the 15,000 records.

Session 3: Mathematical Implementation of the Simulator Algorithm

User:

I am designing the core logic for the /analytics/simulator endpoint. I want to build a deterministic Rule Engine in Java based on the precise physiological heuristics **provided by the dataset author**.

According to the CWK1 specifications, the underlying mathematical model involves:

3. A base signal fraction: Calories / (Steps + 20 * ActiveMinutes).
4. Linear adjustments for static traits (muscle mass, hydration, etc.).
5. Cardiovascular scalers acting as multipliers (e.g., 80 / RestingHR).
6. Strict behavioral constraints (e.g., a 25% penalty for sleep < 5 hours, and habit jumps for streaks).
7. A numerical boundary limit.

How can I architect this mathematically in Java so it's clean, avoids floating-point precision loss, and doesn't just devolve into a massive, unmaintainable 500-line if-else block?

AI:

Excellent approach. Translating the author's complex physiological specifications into a deterministic, in-memory Rule Engine is a highly advanced architectural decision that will give your API true prescriptive value.

To prevent the "spaghetti code" of massive if-else blocks and manage floating-point precision, you should implement a **Sequential Pipeline Pattern** using Java's double for performance, combined with strict phase isolation.

Instead of evaluating everything at once, pass a score variable through distinct mathematical

phases. Here is the architectural blueprint aligning with the author's 5-step heuristic:

5. **Phase 1: Base Signal Evaluation** (The core fraction calculation)
6. **Phase 2: Linear Weights** (Addition/Subtraction for static traits like Muscle Mass and Hydration)
7. **Phase 3: Non-Linear Multipliers** (Applying Heart Rate scalers like 80 / RestingHR)
8. **Phase 4: Consistency & Penalties** (Applying streak multipliers and severe sleep penalties)
9. **Phase 5: Boundary Enforcement** (Clamping the final result to valid categorical tiers)

Here is a conceptual Java structure to keep the math clean and sequential:

Java

```java
public double calculateSimulatedEfficiency(SimulationRequestDTO request) {
    // Phase 1: Base Signal
    double activeWeight = request.getActiveMinutes() * 20.0;
    double score = request.getCaloriesBurned() / (request.getStepsPerDay() + activeWeight);

    // Phase 2: Linear Weights
    score += 0.3; // Example: Muscle Mass adjustment
    score += (request.getHydrationLiters() * 0.05);

    // Phase 3: Non-Linear Multipliers
    score *= (80.0 / request.getHeartRateResting());

    // Phase 4: Consistency & Penalties (Behavioral Constraints)
    if (request.getSleepHours() < 5.0) {
        score *= 0.75; // -25% Severe Deprivation drop
    }
    if (request.getContinuousExerciseDays() >= 5) {
        score += (score * 0.10); // +10% Adaptation Jump
    }

    // Phase 5: Boundary Enforcement
    return Math.min(10.0, Math.max(0.0, score));
}
```

User:

This 5-phase isolation is exactly what I needed. It keeps the mathematical pipeline completely predictable. One detail: the author also explicitly mentioned that the dataset contains about 8% deliberate label noise. Since my simulator strictly implements their exact 5 rules, how should I address this noise?

AI:

That is a critical nuance. Your simulator *should* remain perfectly deterministic according to the

author's rules. Because you have modeled their exact physiological heuristic, your Rule Engine effectively acts as the Ground Truth.

You do not need to inject random noise into your simulator. Instead, you can use your deterministic Rule Engine as an **Anomaly Detection Engine**. If a real historical record from the database deviates from your Rule Engine's calculated output by more than a certain threshold, you can flag that row as part of the "8% deliberate noise." This proves you not only built a simulator but also thoroughly understood the author's dataset specifications.

Session 4: Professional API Documentation Engineering

User:

I am finalizing the MetaboTrack API and want to ensure the deliverables meet a publication-quality standard. I am integrating Springdoc OpenAPI (Swagger 3). My goal is to make the Swagger UI fully interactive so that examiners can test endpoints directly from the browser without needing Postman.

However, the auto-generated Swagger documentation often lacks context, and the default JSON payloads are either empty or confusing. How can we engineer the documentation to lower the barrier to entry for users?

AI:

To elevate your API documentation from a simple reference manual to an interactive testing suite, you need to focus on Developer Experience (DX). Relying solely on auto-generation is insufficient for enterprise-grade APIs.

You need to explicitly define structural schemas and inject ready-to-use, highly realistic JSON payloads. We can achieve this by systematically applying the @Schema and @Operation annotations. By providing context-rich descriptions and exact data examples, users can simply click "Try it out" and immediately execute successful requests.

User:

Exactly. Let's take two specific examples where default Swagger generation fails.

First, my SimulationRequestDTO has many physiological parameters.

Second, my batch delete endpoint accepts a List<Long> as the @RequestBody. Swagger usually struggles to render a JSON array example properly for List parameters, often just showing an empty string box.

Can you help me formulate the precise @Schema(example="...") implementations to fix these rendering issues?

AI:

Certainly. Let's tackle both scenarios.

Scenario 1: Granular Schema for Complex DTOs

For the SimulationRequestDTO, we will apply field-level @Schema annotations with realistic physiological examples. This ensures the auto-generated JSON payload in the Swagger UI is fully populated and mathematically sound for your simulator.

Java

```java
@Schema(description = "Data Transfer Object for Metabolic Simulation Request")
public class SimulationRequestDTO {

    @Schema(description = "Daily step count", example = "12500")
    private Integer stepsPerDay;

    @Schema(description = "Active exercise duration in minutes", example = "45")
    private Integer activeMinutes;

    @Schema(description = "Resting heart rate in BPM", example = "62.5")
    private Double heartRateResting;

    // ... other fields
}
```

Scenario 2: Bypassing Collection Rendering Issues

For the List<Long> batch delete payload, you are correct that standard generation often fails to render a proper JSON array. To force Swagger to render an interactive, formatted JSON array, you must inject a hardcoded JSON string directly into the parameter's @Schema annotation within the Controller.

Here is the exact formulation:

Java

```java
@DeleteMapping
@Operation(summary = "Batch Delete Records", description = "Hard delete multiple records simultaneously.")
public Result<Boolean> deleteRecords(
        @Parameter(description = "List of record IDs to be deleted")
        @Schema(example = "[\n  1001,\n  1002,\n  1005\n]")
        @RequestBody List<Long> ids) {
    // ...
}
```

By explicitly injecting [\n 1001,\n 1002,\n 1005\n], the Swagger UI will parse the string and render a perfectly formatted, clickable JSON array in the request body box. This creates a frictionless, documentation-driven testing environment.